Advanced Database Systems - Lecture Notes
Unit 3: Normalization

Normalization is the process of organizing the attributes and relations of a relational database to reduce data redundancy and improve data integrity. A poorly designed relation suffers from update anomalies: an insertion anomaly occurs when we cannot record a fact without also recording an unrelated fact, a deletion anomaly occurs when deleting one fact unintentionally removes another, and a modification anomaly occurs when the same fact is stored in many rows and must be updated everywhere.

Normalization works by decomposing relations into smaller relations based on functional dependencies. A functional dependency X -> Y holds when each value of X determines exactly one value of Y. The normal forms 1NF, 2NF, 3NF and BCNF are progressively stricter conditions on these dependencies.

First Normal Form (1NF)

A relation is in first normal form when every attribute contains only atomic (indivisible) values and there are no repeating groups. Each row must be identifiable by a primary key.

Example: a table STUDENT(StudentID, Name, Courses) where Courses holds the list "DBMS, Networks, AI" violates 1NF because the Courses attribute is multi-valued. To convert it to 1NF we create one row per student and course: ENROLLMENT(StudentID, Name, Course) with rows (101, Asha, DBMS), (101, Asha, Networks) and (101, Asha, AI). Now every attribute value is atomic.

Second Normal Form (2NF)

A relation is in second normal form when it is in 1NF and every non-key attribute is fully functionally dependent on the whole primary key. 2NF only matters when the primary key is composite. A partial dependency exists when a non-key attribute depends on only part of a composite key.

Example: ENROLLMENT(StudentID, CourseID, StudentName, CourseTitle, Grade) has the composite key (StudentID, CourseID). StudentName depends only on StudentID and CourseTitle depends only on CourseID, which are partial dependencies. Decompose into STUDENT(StudentID, StudentName), COURSE(CourseID, CourseTitle) and ENROLLMENT(StudentID, CourseID, Grade). Each relation is now in 2NF.

Third Normal Form (3NF)

A relation is in third normal form when it is in 2NF and no non-key attribute is transitively dependent on the primary key. A transitive dependency exists when a non-key attribute depends on another non-key attribute, for example A -> B and B -> C, so A -> C.

Example: EMPLOYEE(EmpID, EmpName, DeptID, DeptName). EmpID -> DeptID and DeptID -> DeptName, so DeptName is transitively dependent on EmpID. Decompose into EMPLOYEE(EmpID, EmpName, DeptID) and DEPARTMENT(DeptID, DeptName). Changing a department name now requires updating a single row, which removes the modification anomaly.

Boyce-Codd Normal Form (BCNF) is a stronger version of 3NF: for every non-trivial functional dependency X -> Y, X must be a superkey.

Unit 4: Transactions and Concurrency Control

A transaction is a logical unit of work that must satisfy the ACID properties. Atomicity means all operations of a transaction complete or none do. Consistency means a transaction moves the database from one valid state to another. Isolation means concurrent transactions do not interfere with each other. Durability means committed changes survive system failures, typically by writing to a write-ahead log.

Concurrency control prevents problems such as lost updates, dirty reads and unrepeatable reads. Two-phase locking (2PL) guarantees conflict serializability: in the growing phase a transaction acquires locks and releases none, and in the shrinking phase it releases locks and acquires none. Strict 2PL holds all exclusive locks until commit, which also prevents cascading aborts.

Unit 5: Indexing

An index is an auxiliary structure that speeds up data retrieval at the cost of extra storage and slower writes. A B+ tree index keeps keys sorted in a balanced tree whose leaves are linked, so it supports both equality lookups and range queries efficiently in O(log n) time. A hash index maps keys to buckets with a hash function and is very fast for equality searches but cannot answer range queries.

A clustered index determines the physical order of rows in the table, so a table can have only one clustered index. A non-clustered (secondary) index stores pointers to rows and a table may have many. Choosing indexes requires balancing query performance against insert and update overhead.